MODULE 3

Storage class

What is storage class

- There are two different ways to characterize a variable.
  - Data type
  - Storage class
- Data type refers to `int, float or char`
- Storage class are used to describe the features of a variable
- It defines the scope (visibility) and lifetime of variables

Storage class

It tells the compiler

- The storage area (memory location) of the variable
- The initial value of the variable ….if not initialized
- The scope of the variable (within a function or within a `for`/`if` block)
- The life time of the variable

Types of storage class

Scope of variable:

It indicates the region/block over which the variable's declaration has an effect or in other words that particular variable can be used.

Lifetime of variable:

It refers to the period during which a variable retains a given value during the execution of a program. It tells how long a variable would be *active* in the program.

Types of storage class

Scope of variable: It indicates the region over which the variable's declaration has an effect or in other words that particular variable can be used.

There are four types of storage class

- Automatic storage class (`auto`)
- Static storage class (`static`)
- External storage class (`extern`)
- Register storage class (`register`)

Different storage classes

Class	Storage	Scope	Default (Initial) Value	Lifetime
`auto`	RAM	Local	Garbage Value	Within a function
`extern`	RAM	Global	Zero	Till the main program ends. One can declare it anywhere in a program.
`static`	RAM	Local	Zero	Till the main program ends. It retains the available value between various function calls.
`register`	Register	Local	Garbage Value	Within the function

Automatic storage class

- Keyword is **`auto`**
- Variable declared inside a function has by default automatic storage class, if it is not explicitly specified.
- They are local to the function or block (loop) in which they are declared
- They can be only accessed within the block/ function they have been declared and not outside them
- It can be assigned initial value. If not, it assumes garbage value.

WAP to show the working of auto storage class

```
#include<stdio.h>
void main()
{
 int x = 10;
 printf("initial x = %d\n", x);
    {
    int x = 20;
    printf("Value of x in block = %d\n",x);
    }
 printf("Value of x after block = %d", x);
}
```

The declaration syntax:
`int x` or `auto int x;`

Output

initial x = 10
Value of x in block = 20
Value of x after block = 10

External storage class

- Keyword is **`extern`**
- These variables are available to all the functions
- Their scope extends from the point of definition through out the entire program
- They are declared outside the function body
- The main purpose of using extern variables is that they can be accessed between two different files which are part of a large program.

WAP to show the working of extern storage class

```
include<stdio.h>
void display();
extern int y;
int y = 40;
void main()
{
  int y = 50;
  printf("Value of y = %d\n", y);
  display();
}
void display()
{
  printf("Value of y = %d\n", y);
}
```

The declaration syntax:
`extern int y;`

Output

Value of y = 50

Value of y in function = 40

Static storage class

The declaration syntax:
`static int x;`

- Keyword is **`static`**
- It may be either internal or external depending on the place of declaration
- They are defined within individual functions and therefore have the same scope as automatic variables.
- If a function is exited and then re-entered at a later time, the static variables defined within that function will return their former values.

WAP to show the working of static storage class

Static

```
#include<stdio.h>
void incr();
int main()
{
  incr();
  incr();
  incr();
}
void incr()
{
  static int i = 1;
  printf("i = %d\n",i);
  i=i+1;
}
```

Output
i = 1
i = 2
i = 3

Auto

```
#include<stdio.h>
void incr();
int main()
{
  incr();
  incr();
  incr();
}
void incr()
{
  auto int i = 1;
  printf("i = %d\n",i);
  i=i+1;
}
```

Output
i = 1
i = 1
i = 1

Register storage class

- Keyword **`register`**
- advises the compiler to store the variable in the CPU's register instead of RAM for faster access.
- However if a register is not free, `auto` storage class is used
- variable can have a maximum size equal to the register size, else `auto` is assumed
- CPU registers are accessed by the computer very fast and so the program is executed very fast. So it is best used for loop counters

WAP to show the working of register storage class

```
#include<stdio.h>
void func();
int main()
{
    func();
    func();
    return 0;
}
void func()
{
    register int x = 1;
    printf("x = %d\n", x);
    x = x+1;
}
```

The declaration syntax:
`register int x;`

Output

x = 1

x = 1

MODULE 3

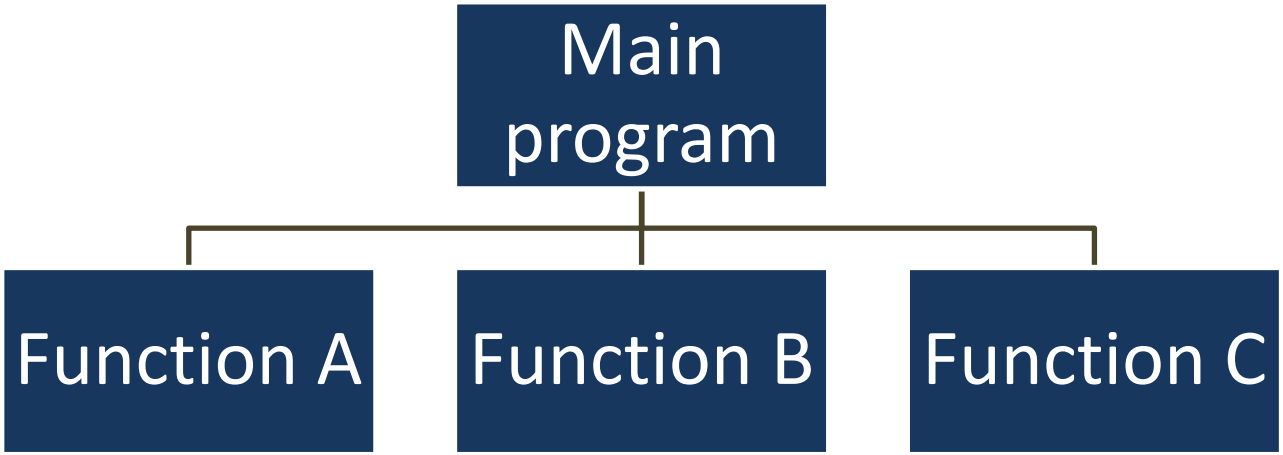
FUNCTIONS

Syllabus

- ✓ **Functions** –
- ✓ Function definition,
- ✓ Function call,
- ✓ Function prototype,
- ✓ Parameter passing;
- ✓ Recursion;
- ✓ Passing array to function;
- ✓ Macros - Defining and calling macros;
- ✓ Command line Arguments.

Functions

- A function is a self contained block of statements that perform a particular task
- Every program must have a main function
- It facilitates top-down modular programming



Modular programming

- It is a strategy applied to the design and development of s/w systems
- Larger programs are divided into program segments called modules
- These modules are integrated to become a s/w system.
- Divide –and-conquer approach to problem solving

Characteristics of modular programming

- ✓ Each module should do only one task
- ✓ Communication between modules is allowed only by the calling module
- ✓ A module can be called by one and only one higher module
- ✓ No communication can take place directly between modules that do not have calling-called relationship
- ✓ All modules are designed as a *single-entry, single-exit* systems using control structures

Advantages of using functions

- It is easy to use
- It reduces the size of the main program
- It is easy to understand the actual logic of the program
- By using functions, it is possible to construct modular and structured program

Built in functions

- These are the library functions
- They are provided by the system and are in library files

Example

- `printf()`
- `scanf()`

User-defined functions

- Defined by the user himself to perform some specific task.
- It provides code reusability and modularity to our program.
- Three elements in user-defined functions
  1. **Function declaration or function prototype**
  2. **Function call**
  3. **Function definition**

```
#include<stdio.h>
int sum_num(int a, int b);
int main()
{
   int a,b, sum;
   a = 5;
   b = 3;
   sum = sum_num(a,b);
   printf("The sum of two numbers is : %d\n", sum);
   return 0;
}

int sum_num(int x,int y)
{
   int z;
   z = x + y;
   return(z);
}
```

Function Declaration

Actual parameters

Function Call

Header
(no semi colon)

Function Definition

Formal parameters

Function declaration

The calling program should declare any function used later in the program.

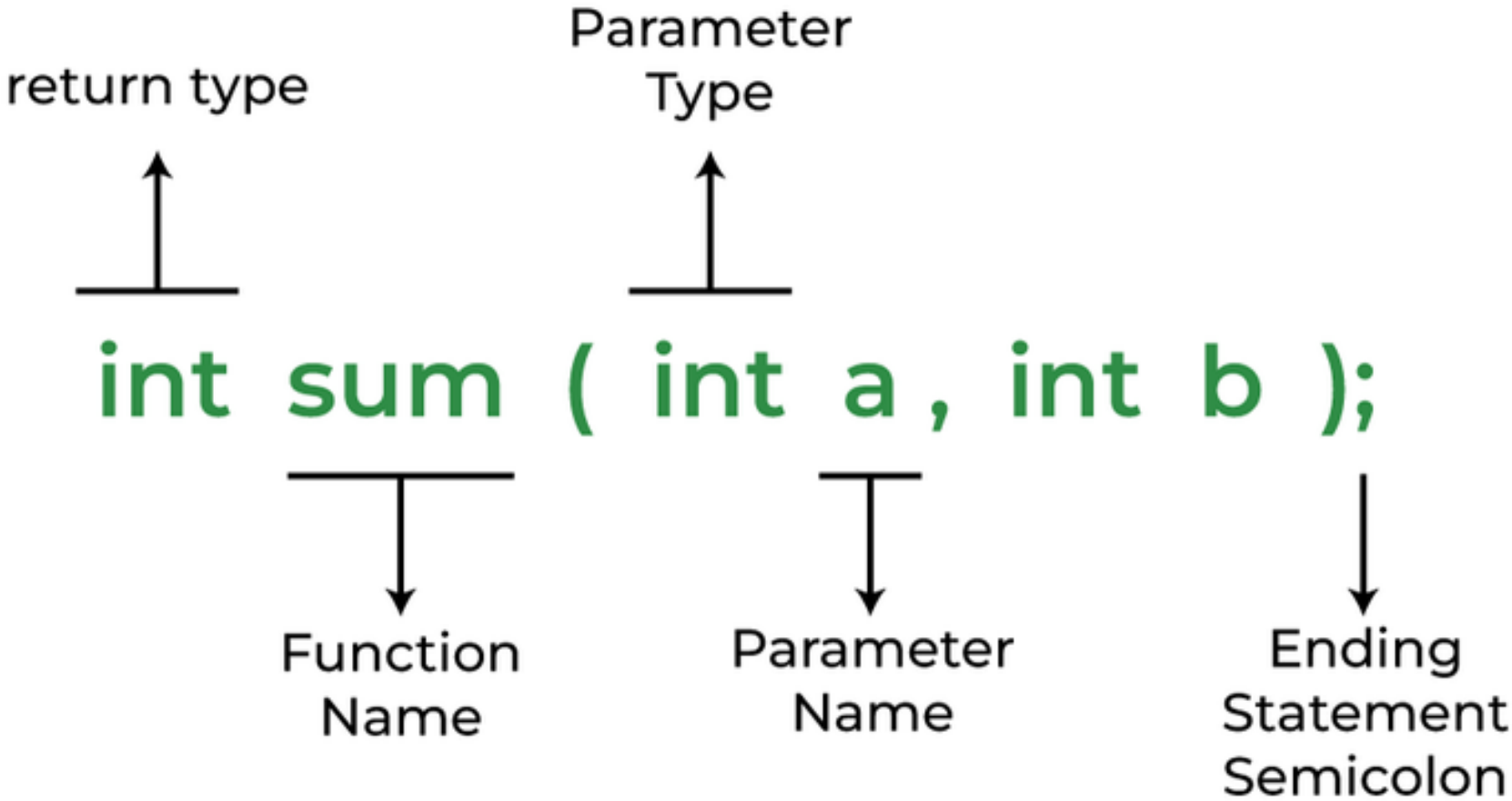
Syntax:

```
return_type name_of_the_function
            (parameter_1, parameter_2);
```

Example:

```
int sum(int a, int b);
float area(float r);
```

Function declaration



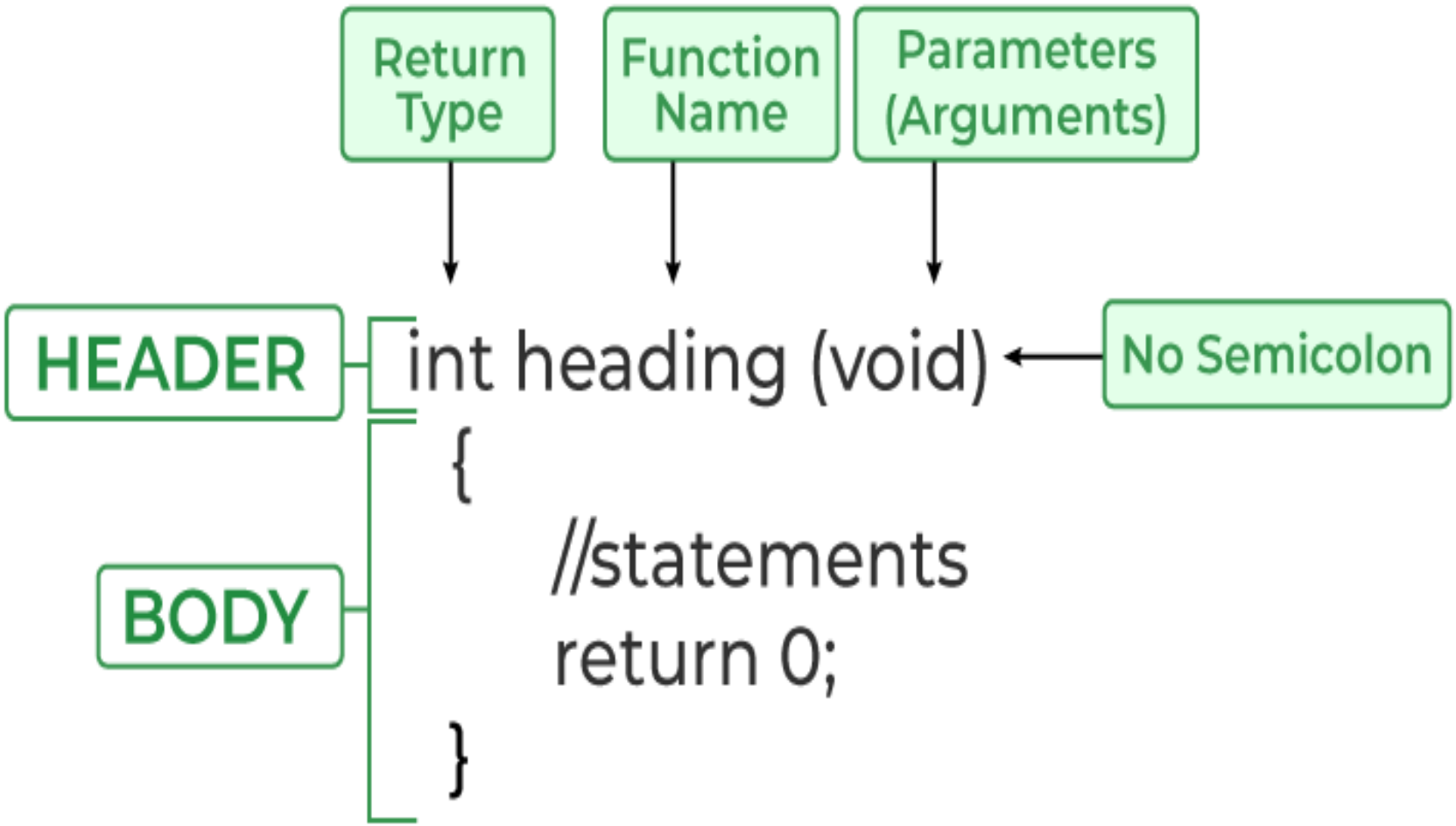
Function definition

The function definition consists of statements which are executed when the function is called

It includes

1. Function name
2. Return type
3. List of parameters
4. Local variable declarations
5. Function statements
6. Return statement

Function definition



Function Definition

Syntax of FUNCTION HEADER:

```
return_type function_name
          (parameter1, parameter2)
```

Example:

```
float interest(float principal, float rate, int rate)
char Grade(int mark_1, int mark_2)
```

Function header has formal parameters

Function call

It is a statement that instructs the compiler to execute the function.

It has function name and actual parameters

Syntax:

```
variable = function_name(list of parameters)
```

Note:

The actual parameters should match with formal parameters.

Categories of function

Functions can be called either **with or without arguments** and might return values. They **may or might not return values** to the calling functions.

- Function with NO arguments and NO return value
- Function with arguments and NO return value
- Function with NO argument, but with return value
- Function with arguments and with return value

Sl. No	Function Type	Syntax for Function Declaration	Syntax for Function Call	Syntax for Function Definition
1.	Functions with no arguments and no return value	void fun_name();	fun_name();	void fun_name() {…. return; }
2.	Functions with arguments but no return value	void fun_name (type1,type2… type n);	fun_name(arg 1, arg2 , … arg n);	void fun_name(type1 arg1, type2 arg2,… typen arg n) {….. return; }
3.	Function with no arguments but with return value	return_type fun_name();	var=fun_name();	return_type fun_name() {…. return value; }
4.	Functions with arguments and with return value	return_type fun_name(type 1, type 2, type n);	var=fun_name(arg 1, arg2,… arg n);	return_type fun_name(type1 arg1, type2 arg2,… type n arg n) {…. return value; }

1. Function with NO arguments and NO return value

Main Prog

```
#include<stdio.h>
void print_string();
int main()
 {
   print_string();
   return 0;
 }
void print_string()
{
 printf("Hello world!\n");
 printf("\"Welcome\"");
}
```

Function Definition

Output:
Hello world!
"Welcome"

1. Function with NO arguments and NO return value

```
#include<stdio.h>
void printsum();
int main()
{
  printsum();
  return 0;
}

void printsum()
{
  printf("The sum of 2 and 3 is %d\t",2+3);
}
```

Output:
The sum of 2 and 3 is : 5

2. Function with arguments and NO return value

```
#include<stdio.h>
void print_sum(int, int);
void main()
{
  int a, b;
  printf("Enter the values of a and b : ");
  scanf("%d %d", &a,&b);
  print_sum(a,b);
}

void print_sum(int x, int y)
{
 printf("The sum of %d and %d is :%d",x, y, x+y);
}
```

Output:

Enter the values of a and b : 3 5

The sum of 3 and 5 is : 8

3. Function with NO arguments but with return value

```
#include<stdio.h>
int get_number();
int main()
{
  int m;
  m = get_number();
  printf("The given number is :%d ", m);
  return 0;
}
int get_number()
 {
  int number;
  printf("Enter the number : ");
  scanf("%d", &number);
  return(number);
 }
```

Output:
Enter the number : 4567
The given number is :4567

4. Function with arguments and with return value

Qn: Maximum of two numbers

```
#include<stdio.h>
int max_num(int, int);
int main()
{
  int a,b, max;
  a = 15;
  b = 69;
  max = max_num(a,b);
  printf("The maximum of two numbers %d and %d is : %d", a,b,max);
  return 0;
}
int max_num(int x, int y)
{
  int m;
  if(x>y)
    m = x;
  else
    m = y;
  return(m);
}
```

Output:

The maximum of two numbers 15 and 69 is : 69

4. Function with arguments and with return value

Qn: Area of circle

```
#include<stdio.h>
float circle_area(int radius);
int main()
{
  int radius;
  float area;
  printf("Enter the radius : ");
  scanf("%d", &radius);
  area = circle_area(radius);
  printf("The area of the circle is : %f", area);
  return 0;
}

float circle_area(int r)
{
  float area;
  area = 3.14*r*r;
  return(area);
}
```

Output:

Enter the radius : 7

The area of the circle is : 153.860

Array as function parameter

Rules to pass array to a function

- The function **declaration** (prototype) must show that the argument is an array
- The function must be called by passing only the name of the array
- In the function **definition**, the formal parameter must be an array type; the size of the array need not be specified

WAP to find the sum of array elements using function

Function declaration

```
#include<stdio.h>
int sum_marks(int marks[],int num);
void main()
{
int result, num;
int marks[5] ={45,38,33,44,50};
num = 5;
result = sum_marks(marks, num);
printf("The sum of marks = %d", result);
}
```

Function call

WAP to find the sum of array elements using function contd

```
int sum_marks(int marks[], int num)
{
  int i,sum;
  sum = 0;
  for(i=0;i<5;i++)
    sum = sum + marks[i];
  return(sum);
}
```

Output:
The sum of marks = 210

Largest element using function

```
#include<stdio.h>
#define MAX 100
int FindMax(int []);
int n;
int main()
{
 int arr[MAX],max, i;
 printf("Enter the no. of elements in array :");
 scanf("%d",&n);
 for(i=0; i<n; i++)
      {
       printf(" a[%d] : ",i);
       scanf("%d",&arr[i]);
      }
```

Function declaration

Largest element using function

Function call

```
max = FindMax(arr);
printf(" Largest in array
            : %d\n", max);
return 0;
}
```

Output:
Enter the no. of elements in array :5
a[0] : 2
a[1] : 20
a[2] : 0
a[3] : -9
a[4] : 67
Largest in array : 67

```
//Function definition//

int FindMax(int arr[])
{
  int i, max;
  i = 1;
  max=arr[0];
  while(i < n)
  {
   if(max < arr[i])
   max = arr[i];
   i++;
  }
  return max;
}
```

Recursion

A function that calls itself is known as a recursive function.

Function Definition

Main function

```
void recurse()
{
    ... .. ...
    recurse();
    ... .. ...
}

int main()
{
    ... .. ...
    recurse();
    ... .. ...
}
```

recursive call

Sum of numbers

10 + sum(9)

10 + (9 + sum(8))

10 + (9 + (8 + sum(7)))

…

10 + 9 + 8 + 7 + 6 + 5 + 4 + 3 + 2 + 1 + sum(0)

10 + 9 + 8 + 7 + 6 + 5 + 4 + 3 + 2 + 1 + 0

WAP to find sum of first 10 numbers using recursive function

```
#include<stdio.h>
int sum(int);
int main()
{
int result;
result = sum(10);
printf("The sum of   numbers
       is : %d", result);
return 0;
}
```

```
int sum(int k)
{
  int sum_num;
  if (k > 0)
  {
   sum_num = k + sum(k-1);
   return sum_num;
  }
  else
  {
    return 0;
  }
}
```

Output:
The sum of numbers is : 55

Factorial

return 5 * factorial(4) = 120

return 4 * factorial(3) = 24

return 3 * factorial(2) = 6

return 2 * factorial(1) = 2

return 1 * factorial(0) = 1

1 * 2 * 3 * 4 * 5 = 120

Find factorial of a number using recursive function

```
#include<stdio.h>
int fact(int);
int main()
{
int x, n;
printf(" Enter the
          Number : ");
scanf("%d",&n);
x = fact(n);
printf("Factorial of
     %d is : %d",n,x);
return 0;
}
```

```
int fact(int n)
{
  int factrl;
  if(n==0)
      return(1);
  else
  {
   factrl= n *fact(n-1);
   return(factrl);
  }
}
```

Output:
Enter the Number : 5
Factorial of 5 is : 120

Macros

Defining and calling macros

Command line Arguments

Macro

- A **macro** is a symbolic name or constant that represents a value, expression.
- They are defined using the #define directive

Syntax

```
#define MACRO_NAME value
```

Macro Examples

```
#define LIMIT 5
#define COUNT 100
#define PI 3.14
#define Max_size 50
#define capital “Delhi”
```

Program using Macro

```
#define PI 3.14159265
int main()
{
double radius = 5.0;
double area = PI * radius * radius;
printf("The area of circle is: %lf", area);
return 0;
}
```

Output:
The area of circle is: 78.539816

Macros without arguments

Macros can replace any code snippet, which makes them extremely versatile

```
#define SQUARE(x) (x * x)
int main() {
int result = SQUARE(5);
printf("The square of 5 is: %d",result);
return 0;
}
```

Output:
The square of 5 is: 25

Macros with arguments

Macros with arguments allow you to create dynamic code replacements by passing values to the macro.

```
#define ADD(x, y) (x + y)
int main()
{
int sum = ADD(3, 7);
printf("The sum of 3 and 7 is: %d", sum);
return 0;
}
```

Output:
The sum of 3 and 7 is: 10

Function like Macros

Macros work in a similar way as a function call. This is known as function-like macros.

Example:

```
#define circleArea(r) (3.1415*(r)*(r))
```

circleArea(5) expands to (3.1415*5*5)

Function like Macros

```
#include <stdio.h>
#define PI 3.1415
#define circleArea(r) (PI*r*r)
int main()
{
float radius, area;
printf("Enter the radius: ");
scanf("%f", &radius);
area = circleArea(radius);
printf("Area = %f", area);
return 0;
}
```

Command in line arguments

Command line Arguments

- It is a parameter supplied to the program when the program is invoked. The parameter may represent a filename the program should process
- Command-line arguments are handled by the `main()` function of a C program
- Mostly `main()` is defined with a return type of `int` and without parameters.
- `main()` can also take two arguments - **argc and argv**

Command line Arguments

Syntax

```
int main(int argc, char *argv[])
```

The variable **argc** is an argument counter that counts the number of arguments on the command line

argv is the argument vector which represents an array of character pointers that points to the command line argument.

The size of this array will be equal to value of `argc`